

Reviewer Pack — Lie Stabilizer Codimension Bounds ABP Width

Entry 82460edbaea0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 20:38:52 UTC

Lie Stabilizer Codimension Bounds ABP Width

Entry ID: 82460edbaea0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 20:38:52 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory: Lie algebra stabilizer of a homogeneous polynomial under the natural GL_N action on $Sym^d C^N$ (Mulmuley-Sohoni, Burgisser-Ikenmeyer style symmetry invariants) **Field B** (complexity object): Algebraic Branching Program (ABP) width for homogeneous multilinear polynomials

Statement:

For a homogeneous polynomial f of degree d in N variables, define $\sigma(f) = \dim\{A \in gl_N : \sum_{i,j} A_{ij} x_j \partial f / \partial x_i \equiv 0 \text{ as a polynomial}\}$, the dimension of the Lie-algebra stabilizer of f under the standard GL_N derivation action. Conjecture: if f is computed by an algebraic branching program of width w (with affine-in-variables edge labels), then $\sigma(f) \geq N - 2w^2$. Equivalently, the GL_N -orbit codimension obeys $\text{codim}(GL_N \cdot f) \geq N - 2w^2$, so any polynomial whose Lie-stabilizer codimension exceeds $N - 2w^2$ requires ABP width strictly greater than w .

Rationale (proposer's reasoning):

GCT's central thesis is that the symmetry (Lie stabilizer) of an explicit polynomial is the obstruction that separates orbit closures; det has a huge Lie stabilizer while permanent's is tiny, mirroring their conjectured complexity gap. Bounded-width ABPs build polynomials by 2×2 -type recursive products and partial sums, which we conjecture force a large 'inherited' continuous symmetry algebra at least linearly in N once w is bounded, whereas random polynomials of degree ≥ 3 generically have $\sigma=0$. The mapping is purely differential-linear-algebraic (kernel of the polynomial-coefficient map on gl_N), so it avoids ring-only constructions that algebrize and is not a truth-table predicate, sidestepping Razborov-Rudich.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d3932ba98776a367

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def fraction_add(f1, f2):
    num1, den1 = f1
    num2, den2 = f2
    common_den = lcm(den1, den2)
    new_num = num1 * (common_den // den1) + num2 * (common_den // den2)
    return (new_num, common_den)

def fraction_sub(f1, f2):
    num1, den1 = f1
    num2, den2 = f2
    common_den = lcm(den1, den2)
    new_num = num1 * (common_den // den1) - num2 * (common_den // den2)
    return (new_num, common_den)

def fraction_mul(f1, f2):
    num1, den1 = f1
    num2, den2 = f2
    return (num1 * num2, den1 * den2)

def fraction_div(f1, f2):
```

```

num1, den1 = f1
num2, den2 = f2
if den2 == 0:
    raise ZeroDivisionError("Fraction division by zero")
return (num1 * den2, den1 * num2)

def fraction_reduce(f):
    num, den = f
    common_div = gcd(num, den)
    return (num // common_div, den // common_div)

def fraction_matrix_mul(A, B):
    rows_A, cols_A = len(A), len(A[0])
    rows_B, cols_B = len(B), len(B[0])
    if cols_A != rows_B:
        raise ValueError("Matrix dimensions do not match for multiplication")

    result = [[(0, 1)] * cols_B for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] = fraction_add(result[i][j], fraction_mul(A[i][k], B[k][j]))

    return result

def build_ABP(N, w, d, p):
    M = [[[random.randint(0, 2) - 1 for _ in range(d + 1)] for _ in range(w)] for _ in range(w)]
    f = [[M[i][j][k] * M[(i + j) % w][k][l] for k in range(d + 1)] for j in range(w) for l in range(w)]

    T = [[[0, 1]] * (N * d) for _ in range(N * d)]
    for i in range(N):
        for j in range(d):
            monomial_index = i * d + j
            for k in range(N):
                T[monomial_index][k] = fraction_add(T[monomial_index][k], (1, 1))

    return f, T

def gaussian_elimination(M):
    rows, cols = len(M), len(M[0])
    rank = 0
    for j in range(cols):
        i_max = rank
        for i in range(rank, rows):
            if abs(M[i][j]) > abs(M[i_max][j]):
                i_max = i

        M[rank], M[i_max] = M[i_max], M[rank]

        if M[rank][j] == 0:
            continue

        for i in range(rank + 1, rows):
            factor = fraction_div((0, 1), (M[i][j], M[rank][j]))
            for k in range(cols):
                M[i][k] = fraction_sub(M[i][k], fraction_mul(factor, M[rank][k]))

```

```

        rank += 1

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    Ns = [8, 10, 12, 14]
    ws = [2, 3]
    ds = [3, 4]

    results = []
    for N in Ns:
        for w in ws:
            for d in ds:
                f, T = build_ABP(N, w, d, 2**30 - 1)

                sigma_f = N * N - gaussian_elimination(T)
                if sigma_f < N - 2 * w ** 2:
                    return {
                        "metric_name": "sigma(f)",
                        "metric_value": sigma_f,
                        "instances_tested": 1,
                        "conjecture_holds": False,
                        "counterexample": f"ABP with N={N}, w={w}, d={d} has sigma(f) < N - 2w^2"
                    }

                results.append(sigma_f)

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = len([x for x in results if x >= N - 2 * w ** 2]) / len(results)

    return {
        "metric_name": "sigma(f)",
        "metric_value": mean,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 35)]

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m

    results = [run_trial(seed) for seed in seeds]
    mean_values = [x["metric_value"] for x in results]
    std_devs = [math.sqrt(sum((y - x["metric_value"]) ** 2 for y in mean_values) / len(mean_values)) for x in results]
    support_fractions = [sum(1 for y in x.values() if y >= N - 2 * w ** 2) / len(x.values()) for x in results]

    total_support_fraction = sum(support_fractions) / len(support_fractions)

```

```

if all(x["conjecture_holds"] for x in results):
    print(f"RESULT: SUPPORTED mean={mean_values[0]:.6f} std={std_devs[0]:.6f} support_fraction={support_fraction}")
elif any(not x["conjecture_holds"] for x in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10694d13.py", line 149, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10694d13.py", line 118, in run_trial
    f, T = build_ABP(N, w, d, 2**30 - 1)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_10694d13.py", line 73, in build_ABP
    f = [[M[i][j][k] * M[(i + j) % w][k][l] for k in range(d + 1)] for j in range(w) for l in range(2)]
          ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unbound local variable 'i' in ABP construction, preventing data collection | next: Fix variable scoping in build_ABP function and rerun tests with validated code

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	239004
2	preregistration	ollama_remote	qwen3:8b	0	0	32844
3	novelty	ollama_remote	qwen3:8b	0	0	28162
4	novelty	ollama_remote	qwen3:8b	0	0	22544
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17592

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21647
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16596
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19083
9	judge	ollama_remote	qwen3:8b	0	0	27356

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 424828 ms total latency. Provider mix: {'claude_max': 1, 'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/82460edbaea0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/82460edbaea0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/82460edbaea0.tar.gz (if generated)